3.1 Introduction

This chapter presents the methodology adopted for the analysis, modeling, and design of the Mwasalati system. It describes the systematic process followed to identify the system requirements, analyze the existing public transportation environment, and develop a technical design that addresses the transportation challenges faced by passengers in Sana'a.

The chapter begins by describing the development methodology and its suitability for the project. It then examines the current transportation situation, identifies the main problems and required improvements, and defines the functional and non-functional requirements of the proposed system. Furthermore, it presents the system analysis and design using appropriate modeling techniques, including UML diagrams and database design, to illustrate the system structure, behavior, and data relationships.

The analysis and design activities provide a foundation for the implementation of Mwasalati, ensuring that the proposed solution remains consistent with the project objectives, scope, and technical architecture.

3.2 Methodology Used

The Mwasalati system adopts an Agile Software Development methodology using the Scrumban framework to manage development activities and support iterative system improvement. This approach combines the flexibility of Agile development with structured task management, allowing the application to be developed through manageable stages. It is suitable for the project because the system involves several interconnected components, including transportation data management, geographic visualization, and route planning. The methodology supports the gradual implementation and refinement of these components while maintaining consistency with the project objectives and technical requirements.

Scrumban combines the iterative development practices of Scrum with the continuous workflow management principles of Kanban. This combination provides flexibility in organizing development tasks, monitoring progress, and adjusting priorities when necessary. In the context of Mwasalati, the approach supports the incremental development of essential functionalities, such as nearby bus line detection, destination search, direct route identification, and transfer-based route planning. It also allows individual components to be evaluated and refined before being integrated into the overall application. Consequently, the development process can accommodate technical challenges and evolving requirements while maintaining a clear focus on the system's intended functionality.

In addition to the development methodology, Object-Oriented Analysis and Design (OOAD) is employed to support the analysis and design of the proposed system. This approach provides a structured means of identifying system components, their responsibilities, and the relationships between them. It is particularly relevant to Mwasalati because the application incorporates distinct functional components that interact to provide transportation guidance. OOAD supports the organization of these components into a coherent design, contributing to system modularity, maintainability, and future extensibility.

The Unified Modeling Language (UML) is used to represent the system's structure, behavior, and interactions through appropriate diagrams. These models provide a clear representation of system requirements and help establish the relationships between users and system functionalities. Furthermore, an Entity-Relationship Diagram (ERD) is employed to represent the database entities, their attributes, and the relationships required to organize transportation information. The ERD supports the design of the Firestore data structure, ensuring that the database model corresponds to the information requirements of the application.

Overall, the integration of Agile development, Scrumban, OOAD, UML, and ERD provides a systematic foundation for the analysis, design, and development of Mwasalati. Each technique contributes to a different aspect of the project, from managing development activities to representing system behavior and organizing transportation data. Together, these appyroaches establish a consistent basis for the subsequent system analysis, requirements specification, and detailed design presented in this chapter.

3.3 System Analysis

This section presents the analysis of the existing public transportation system in Sana’a and identifies the main challenges faced by passengers when using public buses. It examines the current transportation process, the limitations of available information sources, and the difficulties associated with identifying suitable bus routes, boarding and drop-off points, and transfer options. Based on this analysis, the section establishes the foundation for understanding the requirements of the proposed Mwasalati system and explains how the system aims to address the identified challenges through an organized, accessible, and user-friendly digital transportation guidance solution.

3.1 Preliminary Investigation of the Current System (As-Is)

Public bus transportation is an important means of daily mobility in Sana’a. However, passengers may encounter difficulties when attempting to travel to unfamiliar destinations because information about local bus routes is not organized within a

dedicated digital transportation guidance system. The current process largely depends on passengers' personal experience and their ability to obtain transportation information from drivers and passersby.

In the existing situation, passengers must determine which bus line serves their intended destination, identify suitable boarding and drop-off points, and establish whether a transfer between multiple buses is required. Since transportation information is communicated mainly through informal channels, passengers, particularly visitors and individuals unfamiliar with the city, may experience uncertainty when planning their journeys. Furthermore, Google Maps does not provide comprehensive coverage of local public bus routes in Sana'a, limiting its usefulness for passengers who require detailed information about the local transportation network.

The main problems identified in the current system are summarized as follows:

1. Lack of Organized Transportation Information: There is no dedicated digital system that provides comprehensive and structured information about public bus routes in Sana'a.

2. Difficulty Identifying Suitable Bus Lines: Passengers may struggle to determine which bus line serves their intended destinations, particularly when they have limited knowledge of the city.

3. Unclear Boarding and Drop-off Points: The locations where passengers should board or leave a bus are not consistently documented or readily accessible.

4. Difficulty Planning Transfer Journeys: Passengers who need to use multiple bus lines have no systematic method for identifying suitable transfer points and connecting routes.

5. Dependence on Informal Information Sources: Passengers frequently rely on drivers, passersby, and personal experience to obtain transportation directions. This approach may be inconvenient and unreliable, especially for individuals unfamiliar with the local bus network.

6. Time Consumption and Travel Uncertainty: The absence of organized route information may increase the time and effort required to identify suitable transportation options and may lead to inefficient route choices and unnecessary travel costs.

The preliminary investigation highlights the need for a localized transportation guidance system that organizes public bus route information and makes it accessible through a digital platform. These findings provide the basis for analyzing the requirements of the proposed Mwasalati system and identifying how its functionality can address the transportation challenges described above.

3.3.2 Detailed Analysis (TO-BE)

The proposed Mwasalati system addresses the challenges identified in the existing public transportation process by providing a dedicated Android mobile application for public bus transportation guidance in Sana’a. The application organizes local transportation information and presents it through an interactive map and an Arabic user interface, helping passengers identify suitable bus routes and plan their journeys more conveniently.

The main solutions and functionalities provided by the implemented Mwasalati application are described as follows:

1. Nearby Bus Route Detection: The application uses the user's current location to identify and display nearby bus lines, helping passengers discover available public transportation options around them.

2. Destination Search: Users can search for destinations and landmarks within Sana’a. This functionality helps passengers identify transportation options for reaching their intended destinations.

3. Direct Route Planning: The system determines suitable direct bus routes to the selected destination when a single bus line can serve the journey, reducing the need for passengers to rely solely on informal directions.

4. Transfer Route Planning: When a direct route is unavailable, the application supports journeys involving transfers between multiple bus lines. It identifies suitable connecting routes and transfer points to help users understand how to complete their journeys.

5. Boarding and Drop-off Point Identification: The application provides information about appropriate boarding and drop-off points along the selected route, helping users determine where to board a bus and where to leave it.

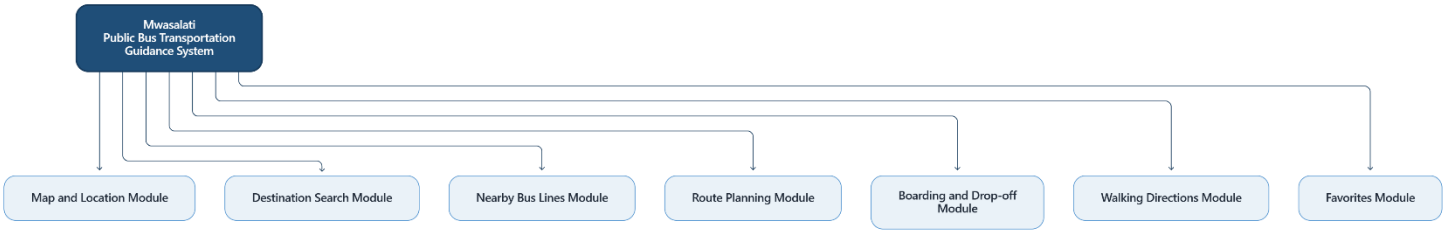
6. Interactive Map and Route Visualization: The application integrates Google Maps to display the user's location, bus routes, and journey paths. Route paths are visualized using map polylines, allowing users to understand the geographic layout of their selected journeys.

7. Arabic User Interface: The application provides an Arabic interface with right-to-left (RTL) support, making transportation information more accessible and understandable to its intended users.

8. Walking Directions to Boarding Points: The system provides walking directions from the user's current location to the appropriate boarding point, helping users reach the starting point of their selected bus journey.

Through these functionalities, Mwasalati transforms locally available public transportation information into a structured and accessible digital resource. The application helps reduce uncertainty when selecting bus routes, identifying boarding

points, and planning journeys that require transfers. Its focus on Sana’a and its local



bus network addresses transportation information needs that are not comprehensively covered by general-purpose navigation applications.

The analysis of the proposed system establishes the functional foundation for the subsequent requirements specification and system design. These requirements and design models will describe the system's expected behavior, data organization, and interaction with users, providing a basis for the implementation and evaluation presented in the following chapters.

3.3.2.1 Decomposition

System decomposition is the process of dividing a system into smaller, manageable modules, each responsible for a specific set of related functionalities. This approach helps organize the system structure, clarify the responsibilities of each module, and simplify the system analysis and design.

The Mwasalati system is decomposed into seven main functional modules based on the implemented features and the requirements of the application. Each module contributes to providing users with public bus transportation guidance within Sana’a.

The main modules of the Mwasalati system are illustrated in Figure 3.X.

Figure 3.: Mwasalati System Decomposition

The seven modules are described as follows:

1. Map and Location Module: Displays the map and the user's current location to support navigation and provide geographical context.
2. Destination Search Module: Enables users to search for destinations and landmarks within Sana’a.
3. Nearby Bus Lines Module: Identifies bus lines located near the user's current location.
4. Route Planning Module: Provides direct and transfer-based public bus routes between the user's location and the selected destination.
5. Boarding and Drop-off Module: Identifies the appropriate boarding and drop-off points along the selected bus route.

6. Walking Directions Module: Provides walking directions from the user's current location to the selected bus boarding point.
7. Favorites Module: Allows users to save and access their preferred routes for convenient future reference.

This decomposition establishes a clear functional structure for the Mwasalati application. It provides a basis for the subsequent system design and analysis by organizing the implemented functionalities into distinct modules according to their responsibilities.

3.3.2.2 Fact-Finding Techniques

To understand the requirements of the Mwasalati system and identify the challenges associated with public bus transportation in Sana'a, several fact-finding techniques were used. These techniques helped gather information about existing transportation conditions, available route data, and the functional requirements of the application.

The following techniques were used:

1. Direct Observation: Direct observation of public transportation and bus routes in Sana'a was conducted to understand the existing transportation environment and the challenges faced by passengers.

2. Review of Available Maps and Information: Available maps and information about bus routes and landmarks in Sana'a were reviewed to support the understanding of geographical locations and transportation routes.

3. Review of Similar Applications: Similar applications and Google Maps were examined to study their available functionalities and limitations, particularly regarding public bus transportation guidance in Sana'a.

4. Practical Application Testing: The Mwasalati application was tested practically to verify the accuracy of bus routes, boarding points, and drop-off points, ensuring that the provided transportation guidance corresponds to the intended functionality.

The information collected through these techniques contributed to identifying the system requirements and guiding the development of the Mwasalati application to address the transportation guidance needs of passengers in Sana'a.

3.3.2.3 User Requirements

This section describes the user requirements of the Mwasalati system, identifying the main functionalities required by passengers and administrators. These requirements define the services provided by the system to support public bus transportation guidance and transportation data management in Sana'a.

1. Passenger Requirements

The passenger should be able to:

1. View their current location on the map.
2. Search for destinations and landmarks within Sana'a.
3. Identify nearby bus lines.
4. Find direct bus routes to a selected destination.
5. Find routes involving transfers between bus lines.
6. Identify boarding and drop-off points.
7. View walking directions to the boarding point.
8. View bus routes on the map.
9. Save and access favorite routes.
10. Use the application in Arabic.

2. Administrator Requirements

The administrator should be able to:

1. Add, update, and delete bus lines.
2. Manage bus routes and boarding and drop-off points on the map.
3. Manage bus and bus line data, including associated colors.
4. Log in securely and manage administrator access permissions.
5. Manage destinations and landmarks.

These user requirements define the main interactions and expected functionalities of the Mwasalati system for both passengers and administrators. They provide a foundation for specifying the system requirements in the following section.

3.3.3 Detailed Analysis (TO-BE)

This section presents the detailed analysis of the proposed Mwasalati system, which aims to address the challenges of public bus transportation in Sana'a. It builds upon the previously identified user requirements and system functionalities to establish a clear foundation for the system design and implementation.

The analysis focuses on defining the functional and non-functional system requirements, describing how the system supports passengers in finding suitable bus routes, identifying boarding and drop-off points, and navigating to their destinations. It also addresses the administrative requirements for managing transportation data, including bus lines, routes, and landmarks.

This section provides the basis for the subsequent system design and UML modeling.

3.3.3.1 Functional Requirements

This section describes the functional requirements of the Mwasalati system. These requirements specify the main operations and services that the system shall provide to passengers and administrators to support public bus transportation guidance and transportation data management in Sana’a.

The functional requirements are divided into two categories: Passenger Functional Requirements and Administrator Functional Requirements.

1. Passenger Functional Requirements

The system shall provide the following functionalities to passengers:

ID / Functional Requirement

FR-01: The system shall display the passenger's current location on the map.

FR-02: The system shall allow passengers to search for destinations and landmarks within Sana’a.

FR-03: The system shall identify nearby bus lines based on the passenger's location.

FR-04: The system shall provide direct bus routes to a selected destination.

FR-05: The system shall provide routes involving transfers between bus lines when a direct route is not available.

FR-06: The system shall display the boarding and drop-off points associated with a selected bus route.

FR-07: The system shall provide walking directions from the passenger's current location to the selected boarding point.

FR-08: The system shall display bus routes and their geographical paths on the map.

FR-09: The system shall allow passengers to save routes as favorites and access their saved routes.

FR-10: The system shall provide an Arabic user interface to support passenger interaction.

2. Administrator Functional Requirements

The system shall provide the following functionalities to administrators:

ID / Functional Requirement

FR-11: The system shall allow administrators to add, update, and delete bus lines.

FR-12: The system shall allow administrators to manage bus routes and boarding and drop-off points on the map.

FR-13: The system shall allow administrators to manage bus and bus line data, including their associated colors.

FR-14: The system shall provide secure administrator login and support access permission management.

FR-15: The system shall allow administrators to manage destinations and landmarks.

These functional requirements define the primary services and operations of the Mwasalati system. They establish a structured basis for the subsequent system design and UML modeling, ensuring that the system's functionality corresponds to the identified user requirements.

3.3.3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational characteristics of the Mwasalati system. They specify the expected standards of usability, performance, reliability, availability, security, compatibility, and data accuracy to ensure that the system provides an effective and dependable public bus transportation guidance service.

1. Usability

The system shall provide a simple, clear, and user-friendly interface that enables passengers to search for destinations, identify nearby bus lines, and plan their journeys with ease. The application shall support the Arabic language and right-to-left (RTL) layout to facilitate its use by the target users.

2. Performance

The system shall provide appropriate response times when displaying maps, searching for destinations, identifying nearby bus lines, and generating direct or transfer routes. The system should process user requests efficiently to reduce waiting time and provide a smooth user experience.

3. Reliability

The system shall provide consistent and dependable results when presenting bus routes, boarding points, and drop-off points. It shall handle errors and unexpected situations appropriately to maintain the reliability of the application's essential functions.

4. Availability

The system shall make its main features accessible when the required internet connection and external services are available. It shall communicate with Firebase Firestore and the necessary mapping services to retrieve and display the information required for route guidance.

5. Security

The system shall protect its stored data and restrict administrative operations to authorized users. It shall provide secure administrator authentication and access permissions to prevent unauthorized access to administrative functions and system data.

6. Compatibility

The system shall be compatible with supported Android devices and accommodate variations in screen sizes and resolutions. Its interface and core functionality shall remain usable across the supported devices.

7. Data Accuracy

The system shall maintain accurate and consistent information about bus lines, routes, destinations, landmarks, and boarding and drop-off points. The accuracy of this information is essential to providing passengers with meaningful route guidance and helping them make informed transportation decisions.

3.3.3.3 System Requirements

This section describes the hardware and software requirements necessary to develop and operate the Mwasalati system. These requirements identify the devices, development tools, and technologies needed to support the application's main functions, including map display, bus route guidance, and data retrieval.

1. Hardware Requirements

The hardware requirements specify the physical devices needed to operate and develop the Mwasalati system.

Hardware / Description

User Devices: Android smartphones equipped with GPS functionality and internet connectivity to support location-based services and map navigation.

Development Computer: A computer used for application development, programming, testing, and maintenance using Flutter and Dart.

2. Software Requirements

The software requirements specify the operating system, development tools, and services used to develop and operate the application.

Software / Technology / Description

Android: The target operating system for running the Mwasalati mobile application.

Flutter: The framework used to develop the mobile application's user interface and functionality.

Dart: The programming language used to implement the application.

Firebase Firestore: The cloud-based database used to store and retrieve application data.

Google Maps API: Used to display geographical maps and bus routes within the application.

Android Studio / Visual Studio Code: Development environments used for writing, building, and testing the application.

Internet Connection: Required to access Firebase Firestore and Google Maps services and retrieve the information needed by the application.

The hardware and software requirements described above provide the technical foundation for developing and operating the Mwasalati system. They support the application's location-based services, geographical visualization, and access to transportation information.

3.3.4 Object-Oriented Analysis (OOA) – UML

Object-Oriented Analysis (OOA) is used to analyze the Mwasalati system by identifying its main actors, functional interactions, activities, and structural components. Unified Modeling Language (UML) diagrams are used to represent the system requirements and behavior in a clear and organized manner.

This section presents the Use Case Diagram, Use Case Specifications, Activity Diagram, Class Diagram, and Sequence Diagram. These models provide a foundation for understanding the system's functionality, interactions, and structure before proceeding to the system design.

3.3.4.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between the Mwasalati system and its primary actors: the Passenger and the Administrator. It provides an overview of the system's main functions and defines the scope of interaction for each actor. The diagram includes passenger functions related to destination search, bus route planning, and map navigation, as well as administrative functions for managing bus lines, routes, and transportation data. This diagram serves as a visual representation of the system's functional requirements and provides a foundation for the subsequent Use Case Specifications.

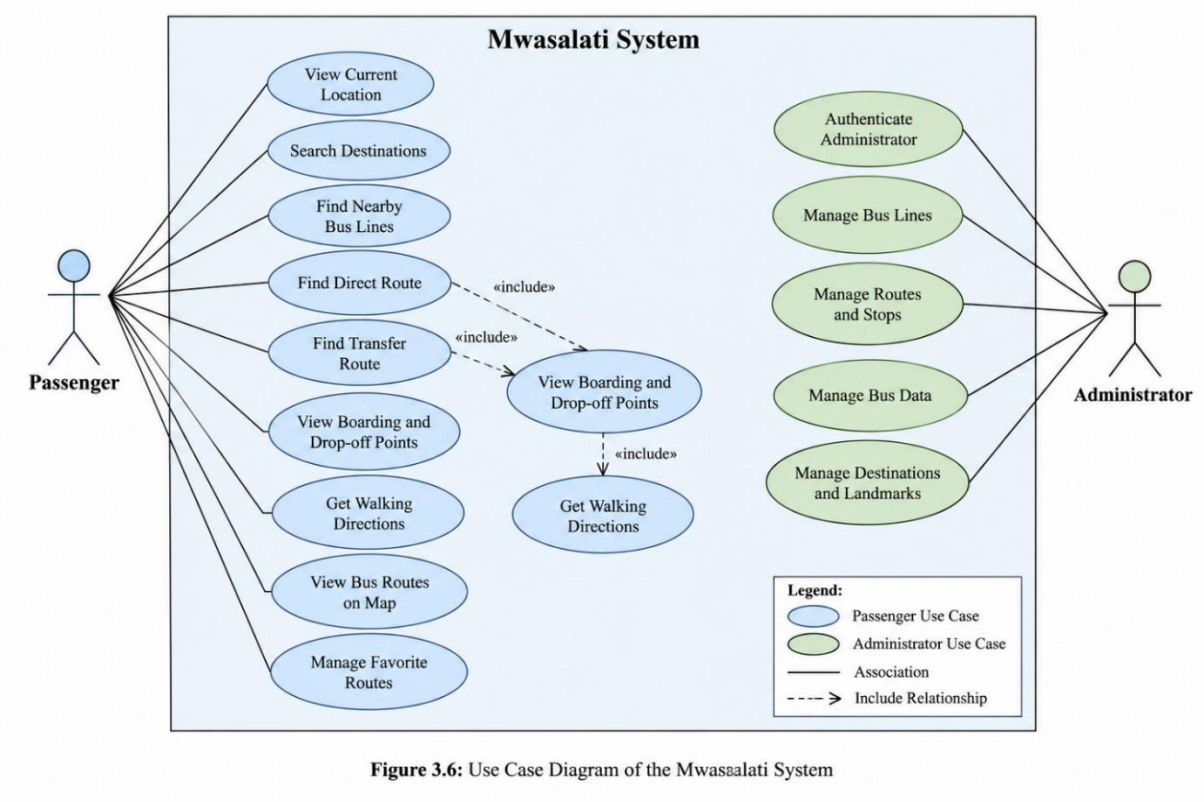


Figure 3.6: Use Case Diagram of the Mwasalati System

3.3.4.2 Class Diagram

The Class Diagram illustrates the structural organization of the Mwasalati system by presenting its main classes, attributes, operations, and relationships. It represents the transportation data, administrative components, and system configuration.

Main Classes

The diagram consists of the following classes:

- City: Represents cities and their geographical information.
- BusLine: Stores bus line information and its associated city.
- Bus: Represents buses associated with bus lines.
- RouteDirection: Represents directional bus routes, including their paths and estimated travel information.
- Landmark: Represents geographical locations and destinations.
- DirectionLandmark: Represents the relationship between route directions and landmarks.
- Admin: Represents administrators and their access information.
- Settings: Stores system configuration parameters.
- TransportVersion: Manages transportation dataset versions.

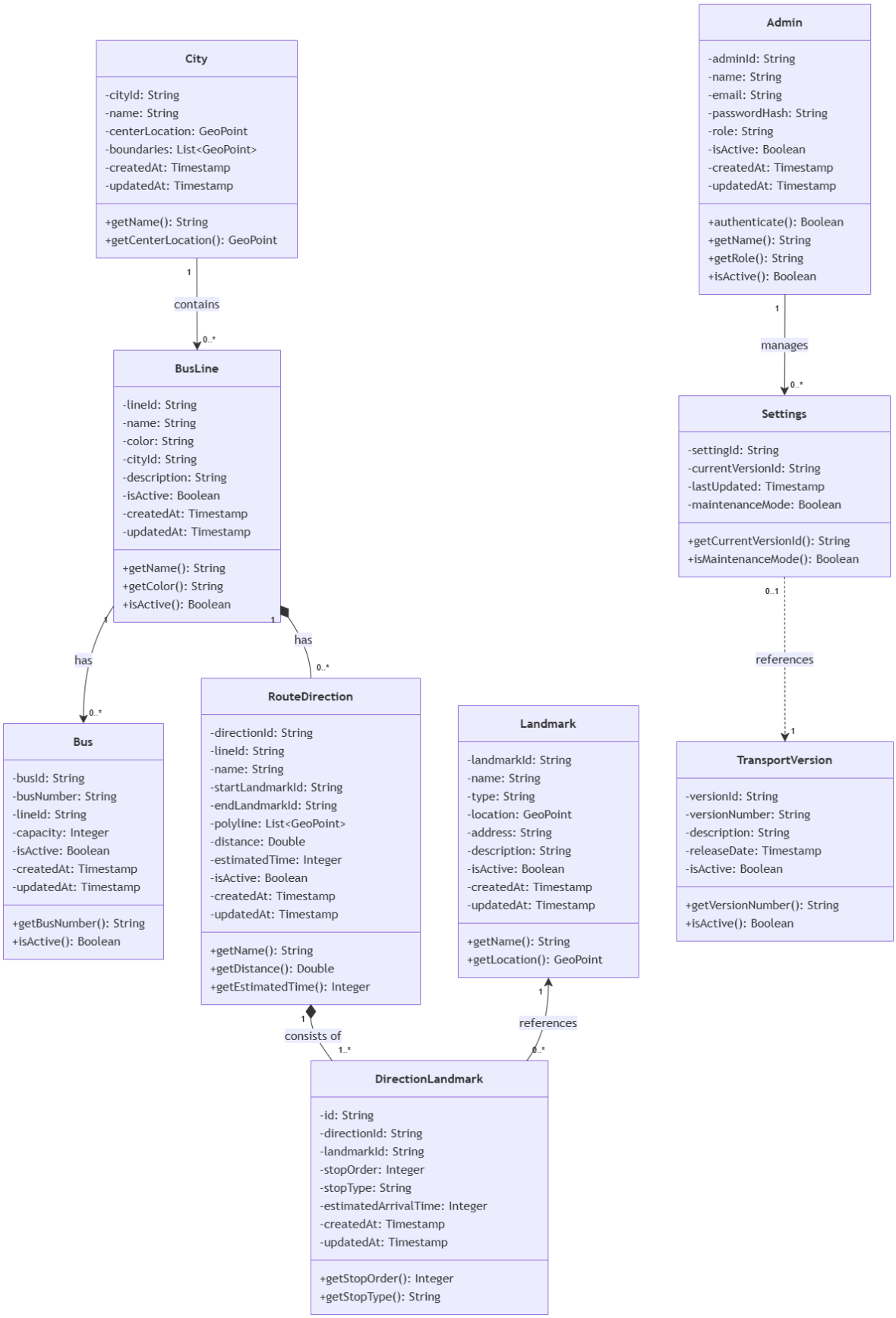
Class Relationships

The diagram illustrates the following relationships:

- A City contains multiple BusLine and Landmark records.
- A BusLine is associated with multiple Bus and RouteDirection records.
- A RouteDirection is associated with multiple landmarks through DirectionLandmark.
- TransportVersion manages transportation data versions, while Settings references the current version.

These relationships define how transportation information is organized and connected within the Mwasalati system.

Overall, the Class Diagram provides a visual representation of the system's structure and serves as a basis for understanding its data organization and implementation.



3.4 System Design

This section presents the system design of the Mwasalati application based on the requirements and analysis described in the previous section. It translates the identified requirements into a structured design that guides system implementation.

The section covers Object-Oriented Design using UML, the Entity-Relationship Model (ERD) for organizing system data, and the interfaces of the application. These design components provide a foundation for developing and understanding the system structure, data relationships, and user interactions.

3.4.1 Object-Oriented Design (OOD) – UML

Object-Oriented Design (OOD) transforms the analysis models of the Mwasalati system into a structured design that supports implementation. UML is used to describe the system's functional interactions and the behavior of its main components.

This section focuses on the Use Case Specifications, which provide detailed descriptions of the system's main use cases, including actors, preconditions, main scenarios, alternative flows, and postconditions. These specifications establish a clear connection between the system requirements and its design.

3.4.1.1 Use Case Specification

This section presents the detailed specifications of selected use cases in the Mwasalati system. Each specification describes the use case name, primary actor, description, preconditions, main success scenario, alternative flows, and postconditions. These specifications illustrate how passengers interact with the system to access bus route information and how administrators manage bus lines. They provide a clear description of the system's expected behavior and support the understanding of its functional requirements.

Use Case Specification – UC-04

Use Case 1

Find Bus Route (Direct or Transfer)

Field / Description

Use Case ID: UC-04

Use Case Name: Find Bus Route (Direct or Transfer)

Primary Actor: Passenger

Description: Allows the passenger to find a suitable bus route from the current location or selected origin to a destination, either directly or through transfers.

Preconditions

- The passenger has opened the Mwasalati application.
- The required transportation data is available.
- The passenger has selected a destination within Sana’a.

Main Success Scenario

1. The passenger opens the route-planning function.
2. The passenger selects the desired destination.
3. The system obtains the passenger's current location or selected origin.
4. The system searches for a direct bus route to the destination.
5. If a direct route is available, the system displays the route and its associated information.
6. If a direct route is unavailable, the system searches for a suitable route involving transfers.
7. The system displays the available route information, including the bus lines and relevant route paths.
8. The passenger reviews the displayed route.

Alternative Flows

- A1 – Direct Route Unavailable: If no direct route is found, the system searches for a route involving transfers.
- A2 – No Suitable Route: If neither a direct nor a transfer route is found, the system informs the passenger that no suitable route is available.
- A3 – Location Unavailable: If the passenger's current location cannot be obtained, the passenger must provide a suitable origin to continue route planning.

Postconditions

- If a suitable route is found, the system displays the route information to the passenger.
- If no suitable route is found, the passenger is informed that a route could not be identified.

UC-06: View Boarding and Drop-off Points

Use Case Name: View Boarding and Drop-off Points

Description: Allows the passenger to view the boarding and drop-off points associated with a selected bus route on the map.

Primary Actor: Passenger

Preconditions:

- The application is open.
- The passenger has selected a bus route.
- The route data is available in the system.

Main Success Scenario:

1. The passenger selects a bus route.
2. The system retrieves the boarding and drop-off points associated with the selected route.
3. The system identifies the locations of the boarding and drop-off points.
4. The system displays the boarding and drop-off points on the map.
5. The passenger reviews the displayed points to identify the appropriate boarding and drop-off locations.

Alternative Flows:

- A1 – Route data unavailable: If the route data cannot be retrieved, the system informs the passenger that the boarding and drop-off information is unavailable.
- A2 – No points available: If no boarding or drop-off points are associated with the selected route, the system informs the passenger that no points are available.

Postconditions:

- The boarding and drop-off points are displayed on the map if the information is available.
- Otherwise, the passenger is informed that the requested information cannot be displayed.

UC-12: Manage Bus Lines

Use Case Name: Manage Bus Lines

Description: Allows the administrator to add, update, and delete bus lines and manage their associated information in the system.

Primary Actor: Administrator

Preconditions:

- The administrator is authenticated.
- The administrator has the required permissions.
- The system is connected to the database.

Main Success Scenario:

1. The administrator accesses the bus line management section.
2. The system displays the available bus lines.
3. The administrator selects an operation: add, update, or delete a bus line.
4. The administrator enters or modifies the required bus line information.
5. The system validates the entered information.
6. The system saves the changes to the database.
7. The system displays a confirmation message.

Alternative Flows:

- A1 – Invalid information: If the entered information is invalid or incomplete, the system informs the administrator and requests corrections.
- A2 – Bus line not found: If the selected bus line does not exist, the system informs the administrator that the bus line cannot be found.
- A3 – Database operation fails: If the system cannot save the changes, it informs the administrator that the operation was unsuccessful.

Postconditions:

- The bus line information is added, updated, or deleted successfully.
- The updated information is stored in the database.
- If the operation fails, the system informs the administrator.

Structured Approach – Entity-Relationship Model (ERD)

The Entity-Relationship Diagram (ERD) represents the logical data structure of the Mwasalati system. It illustrates the main entities, their attributes, primary and foreign keys, and the relationships among the transportation data components. The model provides a structured representation of how cities, bus lines, route directions, landmarks, administrative accounts, system settings, and transportation data versions are organized.

Although Mwasalati uses Firebase Firestore as its database, the ERD is used as a conceptual and logical model to clarify the data structure and relationships before mapping them to the database implementation.

2. Main Entities and Their Descriptions

The logical ERD consists of eight entities, each representing a specific component of the system.

Entity / Description

City: Represents a city supported by the transportation system and serves as the geographical container for bus lines, route directions, and landmarks.

BusLine: Represents a public bus line within a city, including its name, code, route color, and status.

RouteDirection: Represents a specific direction of a bus line, such as GO or BACK, and stores route-related information, including its path, fare, distance, and estimated duration.

Landmark: Represents a geographical landmark or location used in the transportation system, including its name, category, and geographical coordinates.

DirectionLandmark: A logical associative entity that represents the many-to-many relationship between route directions and landmarks.

Admin: Represents an authorized system administrator and stores administrative account information and role-related data.

Settings: Represents the system configuration and stores operational settings, including transfer and walking thresholds, maintenance information, and enabled cities.

TransportVersion: Represents a transportation data version and supports the organization and publication of transportation data.

3. Entity Relationships and Cardinalities

The ERD defines the relationships between the entities using cardinality notation to indicate how many records of one entity can be associated with records of another entity.

3.1 City Relationships

The City entity has one-to-many (1:N) relationships with BusLine, Landmark, and RouteDirection.

- City and BusLine (1:N): One city can contain multiple bus lines, while each bus line belongs to one city.
- City and Landmark (1:N): One city can contain multiple landmarks, while each landmark is associated with one city.
- City and RouteDirection (1:N): One city can contain multiple route directions, while each route direction belongs to one city.

These relationships organize transportation information according to its geographical scope.

3.2 BusLine and RouteDirection

The relationship between BusLine and RouteDirection is one-to-many (1:N). A single bus line can have multiple route directions, such as GO and BACK, while each route direction is associated with one bus line.

The RouteDirection entity stores the details of each direction, including the encoded route polyline, associated landmarks, fare, distance, and estimated travel duration.

3.3 RouteDirection and Landmark

The relationship between RouteDirection and Landmark is many-to-many (N:M). A route direction can be associated with multiple landmarks, and a landmark can be associated with multiple route directions.

This relationship is represented logically through the associative entity DirectionLandmark, which contains the directionId and landmarkId attributes as references to the related entities.

In the Firestore implementation, this relationship is represented using the landmarkIds[] array within RouteDirection, rather than a separate DirectionLandmark collection.

3.4 TransportVersion Relationships

TransportVersion has one-to-many (1:N) relationships with City, BusLine, and RouteDirection.

One transportation data version can be associated with multiple cities, bus lines, and route directions. The versionId attribute is used to associate the relevant transportation records with their corresponding data version.

This structure supports transportation data versioning and publication management.

3.5 Admin and Settings

The Admin entity stores administrator account information, including the administrator's UID, email, role, and status.

The Settings entity stores system-wide configuration values, such as transfer thresholds, walking thresholds, maintenance messages, the current transportation data version, and enabled cities.

These entities are represented independently in the ERD because the documented logical model does not define direct relationships between them and the transportation entities.

4. Keys and Attribute Representation

The ERD uses primary keys (PK) to identify individual entity records and foreign keys (FK) to represent references between related entities.

Primary keys include cityId, lineId, directionId, landmarkId, uid, versionId, and the identifier of the logical associative entity DirectionLandmark.

Foreign-key references include cityId in BusLine and Landmark, lineId and cityId in RouteDirection, and versionId in the transportation entities associated with TransportVersion.

The DirectionLandmark entity uses directionId and landmarkId to reference RouteDirection and Landmark, respectively.

In Firestore, these references are represented through document identifiers and stored fields rather than relational database constraints. Therefore, the ERD illustrates the logical relationships without implying that Firestore enforces traditional relational foreign-key constraints.

5. ERD Diagram

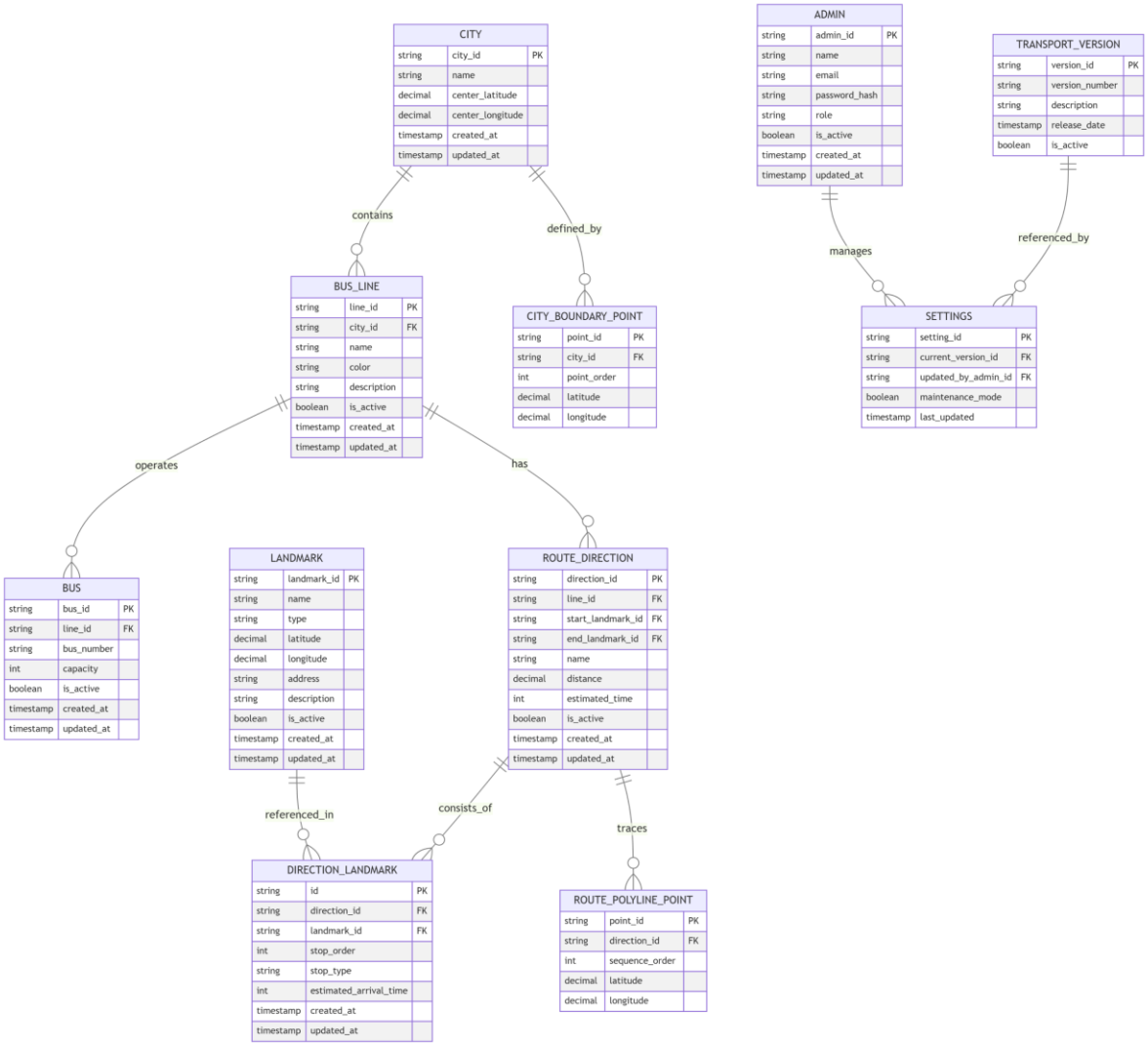
The following figure presents the logical Entity-Relationship Diagram of the Mwasalati system, including its entities, attributes, key references, and relationships.

Insert the approved ERD image here.

Figure 3.8. Entity-Relationship Diagram (ERD) of the Mwasalati System.

6. Summary

The ERD provides a structured overview of the Mwasalati transportation data model. It defines the main entities and their relationships, clarifies the organization of bus lines, route directions, and landmarks, and illustrates the logical association between transportation records and data versions. This model serves as a reference for understanding the Firestore data structure and supports the system's database implementation and future maintenance.



3.4.3 Interfaces

This section presents the main user interfaces of the Mwasalati mobile application. The interfaces are designed to provide passengers with a simple and convenient way to access public transportation information and plan their journeys within Sana'a. The following figures illustrate the main screens of the application, including the Start Screen, Plan Screen, and Favorites Screen, highlighting their primary functions and how they support users in navigating the public bus transportation system.

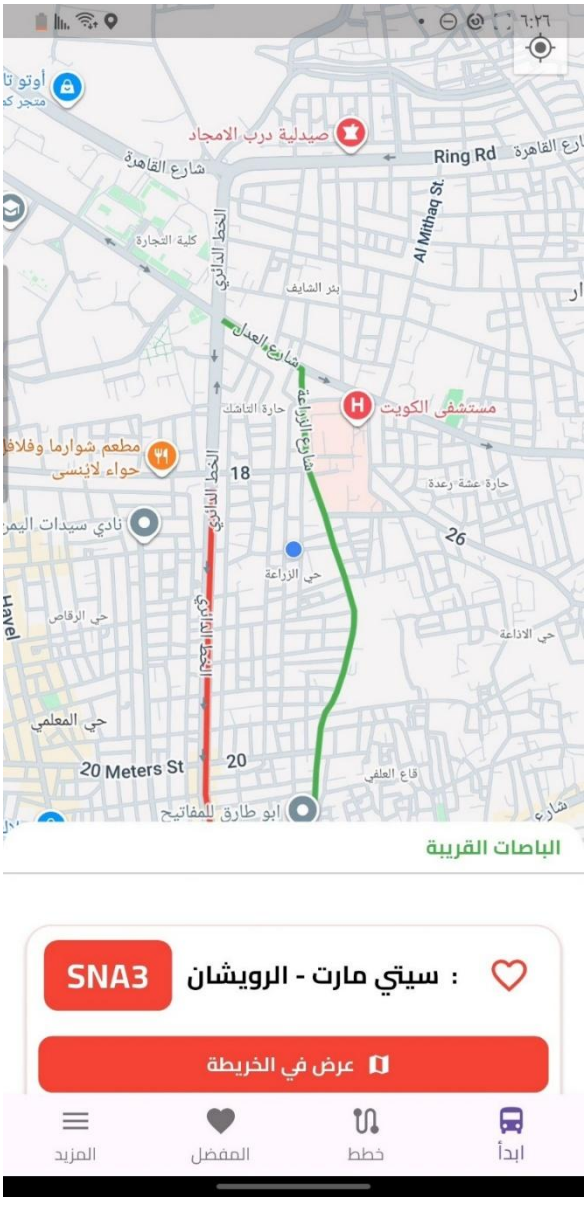
Interface 1: Start Screen (Main Map Interface)

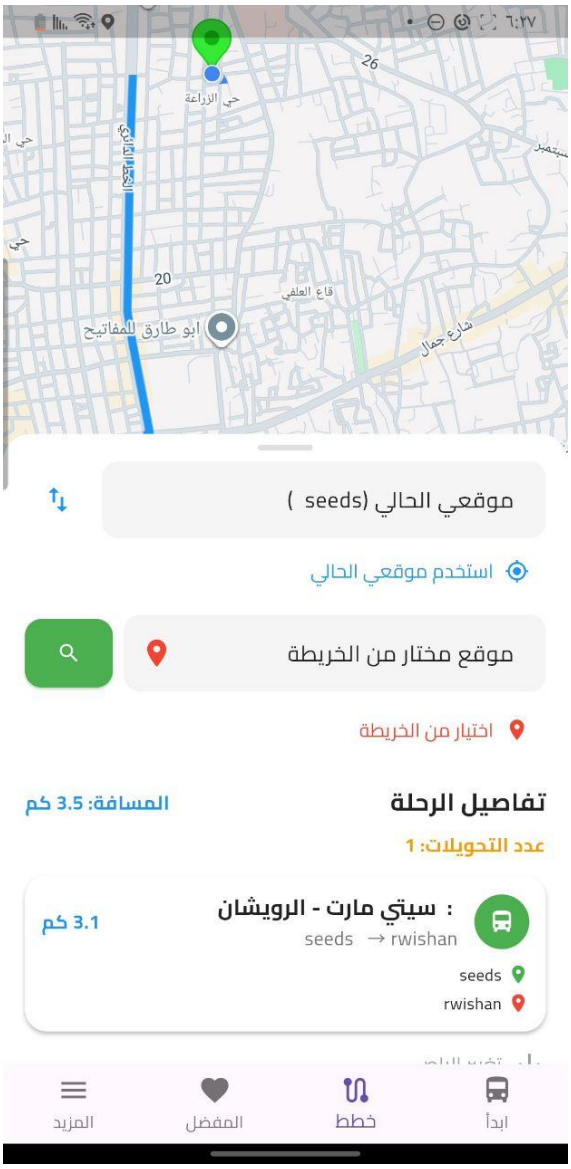
The Start Screen serves as the main map interface of the Mwasalati application. It displays the user's current location on the map and shows nearby bus lines to help passengers identify available transportation options. The interface allows users to explore walking directions to nearby buses through the map and view major landmarks along the bus routes. The map provides a visual representation of the surrounding

transportation network, helping users understand the locations of bus routes and nearby points of interest.

Interface 2: Plan Screen (Route Planning Interface)

The Plan Screen allows passengers to search for a destination and plan their bus journeys. It enables users to specify their starting point and destination, then displays the suggested route on the map. When transfers are required, the interface presents the transfer points and provides the passenger with the complete trip details. This interface helps users understand the proposed journey, including its route and transfer information, before starting their trip.





Interface 3: Favorites Screen

The Favorites Screen allows passengers to access the bus lines they have saved for convenient future reference. The interface displays the saved bus lines along with their identifying information and provides options to view a selected line or remove it from

the favorites list. This feature enables users to quickly access their preferred bus lines without having to search for them again.

